

Doctor Module — Production Frontend Integration Guide

Medical Application | API Contract, Authentication, Authorization, Approval Workflow, and Integration Reference
Version 1.0 — August 2026

Quick Reference

- Identity source of truth: `accounts.User`
- Professional source of truth: `doctors.Doctor`
- Approval source of truth: `Doctor.approval_status`
- Doctor states: pending, approved, rejected, suspended
- Self-deactivation: `POST /doctors/me/deactivate/`
- Doctor deletion: not supported

1. Purpose and Scope

- Manages doctor profiles, specialties, qualifications, availability, account self-service, and administrator approval.
- The custom `accounts.User` model is the identity source of truth. `Doctor` is a professional profile linked one-to-one to `User`. `Doctor` records are preserved rather than deleted.

2. Core Architecture

- `User` = authentication and account identity.
- `Doctor` = professional identity and practice status.
- `Specialty` = medical specialization catalog.
- `Qualification` = education/professional qualification belonging to a `Doctor`.
- `DoctorAvailability` = recurring working-time configuration belonging to a `Doctor`.
- `Doctor` approval is separate from `User.is_active`. A user may authenticate while a doctor application is pending, but clinical doctor capabilities require an approved doctor.

3. Doctor Lifecycle

- New doctor application → PENDING.
- Administrator review → APPROVED, REJECTED, or later SUSPENDED.
- Rejected applications remain stored and may be corrected/resubmitted according to the product workflow.
- Approved doctors can perform approved-doctor operations.
- Suspended doctors retain their records but cannot perform approved-doctor operations.
- Self-deactivation changes `User.is_active` to `False`; it does not delete the `Doctor`.

4. Status Values

- pending — application awaiting administrative review.
- approved — doctor approved to operate.
- rejected — application rejected; `rejection_reason` explains why.
- suspended — administrator has suspended the doctor.

5. Permission Model

- `IsStaffUser` — administrative operations.

- IsDoctor — authenticated, active user with an approved Doctor profile.
- IsDoctorOwner — authenticated user owns the Doctor profile.
- IsApprovedDoctorOwner — authenticated user owns the Doctor profile and is active and approved.
- Because APIView does not automatically invoke object-level permissions, views explicitly call the appropriate object permission checks.

6. Security Principles

- Never accept a user ID from the frontend to establish doctor ownership; the backend uses request.user.
- Never allow the frontend to set approval_status, approved_by, approved_at, rejection_reason, or similar administrative fields through normal profile updates.
- Never expose doctor deletion as a normal operation.
- Normal discovery should expose only approved and active doctors to non-staff users.
- Frontend route guards are UX only; backend permissions are authoritative.
- Use one shared authenticated API client with centralized token expiry/refresh and error handling.

7. API Endpoint Reference

- GET /api/doctors/specialties/ — list specialties.
- POST /api/doctors/specialties/ — create specialty; staff only.
- GET /api/doctors/specialties/{id}/ — retrieve specialty.
- PATCH /api/doctors/specialties/{id}/ — update specialty; staff only.
- DELETE /api/doctors/specialties/{id}/ — soft-deactivate specialty; staff only, provided Specialty has is_active and updated_at.
- GET /api/doctors/doctors/ — list doctors; non-staff normally see approved/active doctors.
- POST /api/doctors/doctors/ — create authenticated user's doctor application; always pending.
- GET /api/doctors/doctors/me/ — retrieve current user's Doctor profile.
- PATCH /api/doctors/doctors/me/ — update current user's editable Doctor fields.
- POST /api/doctors/doctors/me/deactivate/ — deactivate authenticated User account.
- GET /api/doctors/doctors/{id}/ — retrieve doctor subject to visibility rules.
- PATCH /api/doctors/doctors/{id}/ — owner or staff update; administrative fields protected.
- DELETE /api/doctors/doctors/{id}/ — unsupported; expected response 405.
- POST /api/doctors/doctors/{id}/approve/ — staff approval.
- POST /api/doctors/doctors/{id}/reject/ — staff rejection; reason required.
- POST /api/doctors/doctors/{id}/suspend/ — staff suspension.
- GET /api/doctors/doctors/{doctor_id}/qualifications/ — list qualifications.
- POST /api/doctors/doctors/{doctor_id}/qualifications/ — approved owner/staff creates qualification.
- GET/PATCH/DELETE /api/doctors/doctors/{doctor_id}/qualifications/{id}/ — retrieve/update/delete qualification subject to ownership.
- GET /api/doctors/doctors/{doctor_id}/availability/ — list availability.
- POST /api/doctors/doctors/{doctor_id}/availability/ — approved owner/staff creates availability.
- PATCH/DELETE /api/doctors/doctors/{doctor_id}/availability/{id}/ — update/delete availability subject to ownership.

8. Doctor Application Flow

- Authenticate the User first.

- POST the professional application fields to /doctors/doctors/.
- The backend establishes user ownership and forces approval_status to pending.
- Display the returned approval_status.
- Pending: show a review-pending state and hide operational doctor controls.
- Rejected: show rejection_reason where available and allow correction/resubmission if supported.
- Approved: expose doctor management and future clinical/appointment features.
- Suspended: hide operating features and show restricted status.

9. Recommended Frontend State

- authUser — current User/account.
- doctorProfile — current user's Doctor.
- doctorStatus — doctorProfile.approval_status.
- isDoctorApproved — doctorStatus === 'approved' && authUser.is_active.
- isStaff — authUser.is_staff.
- Do not treat a local isDoctor flag as authoritative; the backend response is the final authority.

10. Profile Fields and Protected Fields

- Typical editable fields: specialty, license_number, years_of_experience, consultation_fee, consultation_duration, and bio, according to the final serializer.
- Protected: id, user, approval_status, approved_by, approved_at, rejection_reason, created_at, updated_at.
- Do not retain both verified and approval_status as competing sources of truth. Remove legacy verified if approval_status is adopted.

11. Qualifications

- Qualifications belong to a Doctor and are addressed through nested Doctor URLs.
- Recommended frontend object: id, doctor, degree, institution, year_of_completion, created_at.
- Ownership is enforced by the backend; a doctor_id supplied in a URL is not authorization by itself.

12. Availability

- Availability represents recurring doctor working periods.
- Client-side validation should reject obvious invalid ranges, but server-side validation must remain authoritative.
- Approved owners and authorized staff may manage availability.
- Before production, implement server-side overlap validation and a canonical day representation.

13. Administrator Workflow

- Admin reviews pending doctor applications.
- Admin approves with POST /doctors/{id}/approve/.
- Admin rejects with POST /doctors/{id}/reject/ and a required reason.
- Admin suspends with POST /doctors/{id}/suspend/ and an appropriate reason.
- Approval/rejection/suspension should not be implemented by PATCHing approval_status through the general doctor profile endpoint.

14. Example Payloads

- Doctor application: {"specialty": 2, "license_number": "LIC-12345", "years_of_experience": 7, "consultation_fee": "500.00", "consultation_duration": 30, "bio": "..."}

- Reject: {"reason": "License documentation requires verification."}
- Suspend: {"reason": "Administrative review required."}
- Availability: {"day": "Monday", "start_time": "09:00:00", "end_time": "13:00:00", "is_available": true}

15. Error Handling

- 401 — authentication required or credentials expired.
- 403 — authenticated but not authorized.
- 404 — resource absent or intentionally hidden by visibility rules.
- 405 — unsupported operation, such as Doctor deletion.
- 409 — state conflict such as an existing Doctor profile.
- 400 — validation error; map field errors to frontend forms.
- Do not infer approval from HTTP status; read approval_status.

16. Frontend Feature Structure

- features/doctors/api.ts — typed API functions.
- features/doctors/types.ts — TypeScript models and enums.
- features/doctors/hooks/ — data fetching and mutation hooks.
- features/doctors/components/ — reusable UI.
- features/doctors/screens/ — pages/screens.
- Keep administrative approval controls separate from the general doctor profile editor.

17. Production Finalization Checklist

- Remove legacy verified if approval_status is the source of truth.
- Add Specialty.is_active and updated_at if specialty deletion is soft deactivation.
- Ensure Doctor has unique license_number if required by business rules.
- Use on_delete=PROTECT for historical professional relationships where appropriate; confirm against the final model.
- Implement availability overlap validation.
- Add pagination to list endpoints.
- Add automated tests for ownership, approval transitions, protected fields, deactivation, and visibility.
- Consider throttling sensitive administrative endpoints.
- Consider an immutable DoctorVerificationHistory for auditability.

18. Testing Checklist

- Unauthenticated requests are rejected.
- Duplicate Doctor profile creation is rejected.
- New applications always become pending.
- Client cannot set approval_status to approved.
- Non-staff cannot approve, reject, or suspend.
- Doctor cannot modify another doctor's profile.
- Pending and suspended doctors cannot perform approved-doctor operations.
- Doctor can deactivate their own User account.
- Doctor deletion returns 405.

- Non-staff cannot enumerate hidden pending/rejected/suspended profiles.
- Qualification and availability ownership is enforced.
- Invalid and overlapping availability is rejected server-side.

19. Appointment Module Dependency

- The future Appointment module should use the Doctor approval state rather than duplicating verification logic.
- Doctor-facing clinical/appointment actions should require an active User and approved Doctor.
- Patients should only be offered doctors who are approved, active, and otherwise eligible for booking.
- Doctor deactivation and suspension must be considered when creating new appointments.
- Historical appointments must remain linked to the Doctor even after account deactivation.

Important Implementation Note

This guide reflects the architecture agreed in the conversation. Before calling the module production-ready, the actual models, serializers, permissions, views, URLs, and admin must match this contract. In particular, Specialty must contain `is_active` and `updated_at` if its DELETE endpoint is implemented as soft deactivation, and the final Doctor model should not retain a competing `verified` boolean.